

AI Hardware Compiler

11/10/2025 Design Review

Senior Design:

- Mikhail Golovenchits
- Sahil Patel
- Heng-I Chu

Volunteering:

- Vihaan Chinthakindi

Background

C/C++

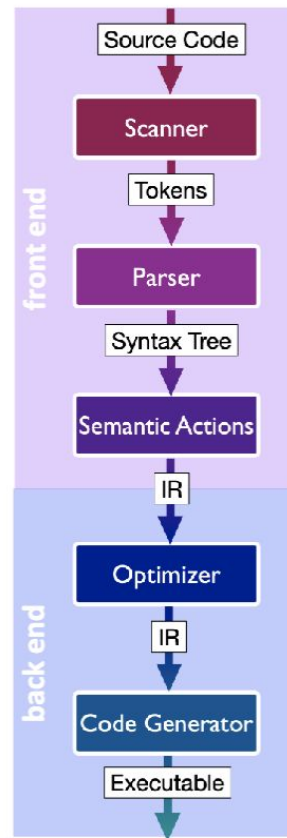
Problem

- The AI Hardware Team's processor currently **requires manual machine code programming**.
- This approach is **time-consuming, error-prone**, and limits developer productivity.
- Developers need a **higher-level programming interface** to streamline development and reduce mistakes.

Solution

- Build a **compiler** that translates C code into custom ISA
- Create an independent software platform to perform calculations on the processor

Atalla ISA

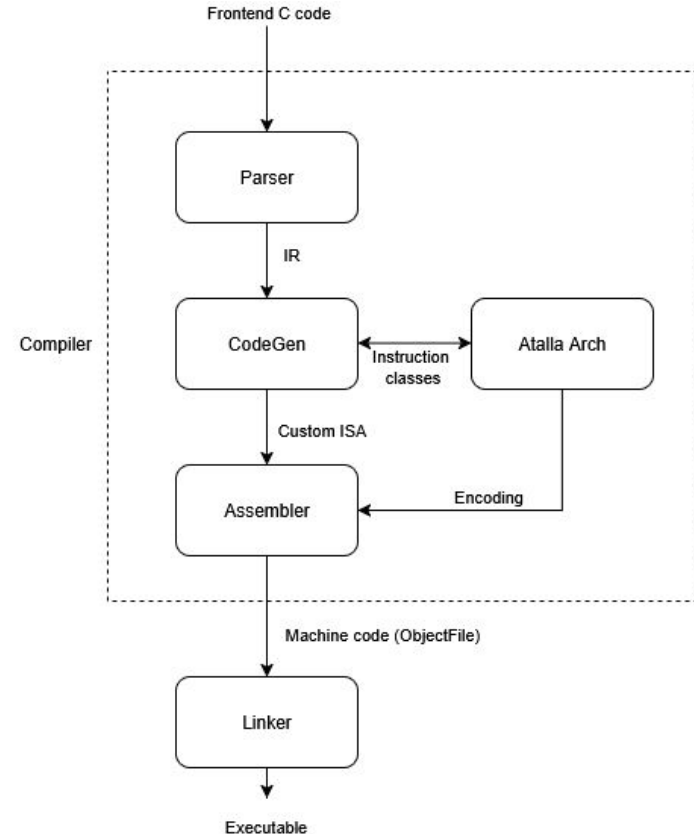


Requirements

- Basic scalar instructions
- Vector data type for referencing specific vectors without direct memory addressing
- Vector backend instructions
- Intrinsic instructions like GEMM & SDMA to perform operations on vectors and bfloat16 arrays
- Very Long Instruction Word Packetization
 - Packing up to 4 instructions into a single vliw for better parallelization
 - Should be dependency and latency aware
- Assembler, linker & executable generator

Work Completed before DR1

- Researched compiler structure (ECE 468/573 + documentation)
 - PPCI chosen over LLVM
- Added POC custom frontend instruction “theta” to parser logic
- Backend Atalla Architecture
 - Tokens & Registers defined
 - Scalar ISA instructions almost completely implemented



Scalar ISA

- The compiler is now fully able to compile basic scalar operations
- Frontend already handled by PPCI
- Changed float datatype to consist of 16 bits to represent bfloat16, in order to reuse existing PPCI float parsing

```
int instruct_tests(int a, int b) {    Sahil Patel, last month • sahil
    int r_acc = 0;

    // R-type: operate purely on registers : add_s, sub_s
    // int r_sum = a + b;
    // int r_diff = a - b;
    // r_acc = r_sum ^ r_diff;

    // I-type: register-immediate arithmetic (addi_s, andi_s)
    int i_adj = a + 4;
    int i_mask = b & 0xFF;
    r_acc ^= i_adj ^ i_mask;

    // Mem-type: explicit loads/stores through stack memory (lw_s, sw_s)
    int mem_buf[2];
    mem_buf[0] = r_acc;
    mem_buf[1] = mem_buf[0] + 1;
    int mem_val = mem_buf[1];

    // BR-type: conditional branch that compares registers (beq_s, blt_s)
    if (mem_val >= i_adj) {
        mem_val -= 2;
    } else if (mem_val == i_adj) {
        mem_val += 3;
    }

    // MI-type: load immediate into a register (li_s)
    // int mi_seed = 0x1234;
    // r_acc ^= mi_seed;

    return r_acc + mem_val;
}
```

```
addi_s vreg0a, x12, 0
addi_s vreg1b, x13, 0
instruct_tests_block0:
jal x0, instruct_tests_block1
instruct_tests_block1:
sw_s vreg0a, -4(x8)
sw_s vreg1b, -8(x8)
li_s vreg19, 0
sw_s vreg19, -12(x8)
lw_s vreg20, -4(x8)
addi_s vreg3tmp_load, vreg20, 0
addi_s vreg21, vreg3tmp_load, 4
sw_s vreg21, -16(x8)
lw_s vreg22, -8(x8)
addi_s vreg4tmp_load_5, vreg22, 0
andi_s vreg23, vreg4tmp_load_5, 255
sw_s vreg23, -20(x8)
lw_s vreg24, -16(x8)
addi_s vreg5tmp_load_10, vreg24, 0
lw_s vreg25, -20(x8)
addi_s vreg6tmp_load_11, vreg25, 0
lw_s vreg26, -12(x8)
addi_s vreg7tmp_load_13, vreg26, 0
xor_s vreg27, vreg5tmp_load_10, vreg6tmp_load_11
xor_s vreg28, vreg7tmp_load_13, vreg27
sw_s vreg28, -12(x8)
lw_s vreg29, -12(x8)
addi_s vreg8tmp_load_19, vreg29, 0
addi_s vreg30, x8, -28
```

Frontend Vector Parsing

- Added “vec” datatype (32*16 bits) to frontend by adding a ir.vec to the IR object.
- Parser successfully detects vector type variables and generates the appropriate IR

```
# The builtin types:
# f64 = FloatingPointTyp("f64", 64) #: 64-bit floating point type
# f32 = FloatingPointTyp("f32", 32) #: 32-bit floating point type
f16 = FloatingPointTyp("f16", 16) #: 16-bit floating point type
i64 = SignedIntegerTyp("i64", 64) #: Signed 64-bit type
i32 = SignedIntegerTyp("i32", 32) #: Signed 32-bit type
i16 = SignedIntegerTyp("i16", 16) #: Signed 16-bit type
i8 = SignedIntegerTyp("i8", 8) #: Signed 8-bit type
u64 = UnsignedIntegerTyp("u64", 64) #: Unsigned 64-bit type
u32 = UnsignedIntegerTyp("u32", 32) #: Unsigned 32-bit type
u16 = UnsignedIntegerTyp("u16", 16) #: Unsigned 16-bit type
u8 = UnsignedIntegerTyp("u8", 8) #: Unsigned 8-bit type
ptr = PointerTyp("ptr") #: Pointer type
vec = VectorTyp("vec", 32*16) #: Vector type
```

IR vector type definitions

```
self.ir_type_map = {
    BasicType.CHAR: (ir.i8, 1),
    BasicType.UCHAR: (ir.u8, 1),
    BasicType.SHORT: (ir.i16, 2),
    BasicType.USHORT: (ir.u16, 2),
    BasicType.INT: (int_types[int_size], int_size),
    BasicType.UINT: (uint_types[int_size], int_size),
    BasicType.LONG: (int_types[long_size], long_size),
    BasicType.ULONG: (uint_types[long_size], long_size),
    BasicType.LONGLONG: (ir.i64, 8),
    BasicType.ULONGLONG: (ir.u64, 8),
    BasicType.FLOAT: (ir.f16, 4),
    # BasicType.FLOAT: (ir.f32, 4), --- IGNORE ---
    # BasicType.DOUBLE: (ir.f64, 8),
    # BasicType.LONGDOUBLE: (ir.f64, 8),
    BasicType.VA_LIST: (ir.ptr, self.ptr_size),
    BasicType.VECTOR: (ir.vec, 64),
}
```

Atalla IR type mapping

Backend Vector ISA

- Created custom vector Instruction, Register, and Token files.
- Specified baseline isa patterns for vectors for assembly generation
- Implemented new tokens based on custom ISA
- Defined custom instructions for vector ISA
- Tested down to selection tree (to assembly in progress)

```
# ----- VV (vector-vector) -----  
  
@isa.pattern("vecreg", "ADDVEC(vecreg, vecreg)", size=2)  
def patt_add_vv(ctx, tree, v0, v1):  
    d = _new_v(ctx)  
    ctx.emit(AddVv(d, v0, v1))  
    return d  
  
@isa.pattern("vecreg", "SUBVEC(vecreg, vecreg)", size=2)  
def patt_sub_vv(ctx, tree, v0, v1):  
    d = _new_v(ctx)  
    ctx.emit(SubVv(d, v0, v1))  
    return d
```

Figure 1

```
# VV  
AddVv = make_vv("add_vv", 0b0101000)  
SubVv = make_vv("sub_vv", 0b0101001)  
MulVv = make_vv("mul_vv", 0b0101010)  
DivVv = make_vv("div_vv", 0b0101011)  
AndVv = make_vv("and_vv", 0b0101100)  
OrVv = make_vv("or_vv", 0b0101101)  
XorVv = make_vv("xor_vv", 0b0101110)  
GemmVv = make_vv("gemm_vv", 0b0101111)  
MgtVv = make_vv("mgt_vv", 0b0110000)  
MltVv = make_vv("mlt_vv", 0b0110001)  
MeqVv = make_vv("meq_vv", 0b0110010)  
MneqVv = make_vv("mneq_vv", 0b0110011)  
  
# VI  
AddVi = make_vi("addi_vi", 0b0110100)  
SubVi = make_vi("subi_vi", 0b0110101)  
MulVi = make_vi("mul_i_vi", 0b0110110)  
DivVi = make_vi("divi_vi", 0b0110111)  
ExpVi = make_vi("exp_i_vi", 0b0111000)  
SqrtVi = make_vi("sqrti_vi", 0b0111001)  
NotVi = make_vi("not_vi", 0b0111010)  
ShiftVi = make_vi("shift_vi", 0b0111011)  
LwVi = make_vi("lw_vi", 0b0111100)  
RsumVi = make_vi("rsum_vi", 0b0111101)  
RminVi = make_vi("rmin_vi", 0b0111110)  
RmaxVi = make_vi("rmax_vi", 0b0111111)  
  
# VS  
ShiftVs = make_vs("shift_vs", 0b0111000)
```

Figure 2

```
class AtallaVToken(Token):  
    class Info:  
        size = 40  
        opcode = bit_range(0, 7)  
        vd = bit_range(7, 15)  
        vs1 = bit_range(15, 23)  
        vs2 = bit_range(23, 31)  
        mask = bit_range(31, 35)  
        sac = bit_range(35, 36)  
        reserved = bit_range(36, 40)  
  
class AtallaVSToken(Token):  
    class Info:  
        size = 40  
        opcode = bit_range(0, 7)  
        vd = bit_range(7, 15)  
        vs1 = bit_range(15, 23)  
        rs1 = bit_range(23, 31)  
        mask = bit_range(31, 35)  
        reserved = bit_range(35, 40)  
  
class AtallaVIToken(Token):  
    class Info:  
        size = 40  
        opcode = bit_range(0, 7)  
        vd = bit_range(7, 15)  
        vs1 = bit_range(15, 23)  
        imm8 = bit_range(23, 31)  
        mask = bit_range(31, 35)  
        imm5 = bit_range(35, 40)  
  
class AtallaVMemToken(Token):  
    class Info:  
        size = 40  
        opcode = bit_range(0, 7)  
        vd = bit_range(7, 15)  
        rs1 = bit_range(15, 23)  
        tile_r_c_count = bit_range(23, 28)  
        rc = bit_range(28, 29)  
        sp = bit_range(29, 31)  
        mask = bit_range(31, 35)  
        rc_id = bit_range(35, 40)
```

Figure 3

GEMM Intrinsic Function Implementation

- GEMM & SDMA instructions need to be implemented using an intrinsic function (i.e. functions the implementation of which is handled by the compiler)
- Vector-Scalar instructions will be handled by overloading the BinOps (+, -, *, /, etc.) in the code generation phase

```
elif self.peek == "gemm":
    location = self.consume("gemm").loc
    self.consume("(")
    args = []
    while self.peek != ")":
        args.append(self.parse_assignment_expression())
        if self.peek != ",":
            self.consume(",")
    expr = self.semantics.on_gemm(args[0], args[1], args[2], location)
    self.consume(")")
```

Parser detection of GEMM

```
def do_gemm(self, node):
    """Create GEMMV node with three u32 register operands.

    This will produce a node like: GEMMV(REGU32(r0), REGU32(r1), REGU32(r2))
    by moving the three input values into fresh u32 vregs, creating REG
    nodes for those vregs and using their outputs as inputs to GEMMV.
    """
    # Gemm.ir stores its operands in node.value as a list
    args = list(node.value)

    reg_outputs = []
    for arg in args:
        # get SGValue for the argument
        arg_val = self.get_value(arg)

        # allocate a new vreg for a u32 register and move the value into it
        vreg = self.new_vreg(ir.vec)
        mov_node = self.new_node("MOV", ir.vec, arg_val, value=vreg)
        self.chain(mov_node)

        # create a REG node that represents the vreg and expose its output
        reg_node = self.new_node("REG", ir.vec, value=vreg)
        out = reg_node.new_output(vreg.name)
        out.vreg = vreg
        reg_outputs.append(out)

    # Create the GEMMV node taking the three register outputs as inputs
    sgnode = self.new_node("GEMMVEC", None, *reg_outputs)
    self.debug_db.map(node, sgnode)
    self.chain(sgnode)
```

IRDAG Selection Tree Generator for GEMM

GEMM & Vector Selection Tree Generation Result

```
int main(){  
    vec v1;  
    vec v2;  
    vec vr;  
  
    gemm(vr, v1, v2);  
}
```

```
=====  
global function i32 main() {  
  main_block0: {  
    blob<64:64> alloca = alloc 64 bytes aligned at 64;  
    ptr alloca_addr = &alloca;  
    blob<64:64> alloca_0 = alloc 64 bytes aligned at 64;  
    ptr alloca_addr_1 = &alloca_0;  
    blob<64:64> alloca_2 = alloc 64 bytes aligned at 64;  
    ptr alloca_addr_3 = &alloca_2;  
    jmp main_block1;  
  }  
  
  main_block1: {  
    gemm ptr alloca_addr_3 = &alloca_2, ptr alloca_addr = &alloca, ptr alloca_addr_1 = &alloca_0;  
    i32 num = 0;  
    return num;  
  }  
}  
=====
```

Selection trees:

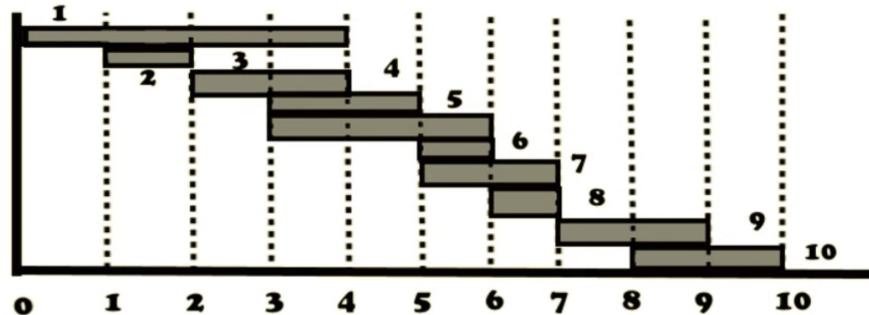
```
main_block0:  
  JMP[main_block1:]  
main_block1:  
  MOVVEC[vreg1](FPRELU32[Stack[64 bytes at -192]])  
  MOVVEC[vreg2](FPRELU32[Stack[64 bytes at -64]])  
  MOVVEC[vreg3](FPRELU32[Stack[64 bytes at -128]])  
  MOVVEC[vreg1](REGVEC[vreg1])  
  MOVVEC[vreg2](REGVEC[vreg2])  
  MOVVEC[vreg3](REGVEC[vreg3])  
  GEMMVEC(REGVEC[vreg1], REGVEC[vreg2], REGVEC[vreg3])  
  MOVF32[vreg0, vreg1](CONSTI32[0])  
  JMP[main_epilog:]  
main_epilog:
```

IR

Selection
trees

VLIW Hardware Constraints

- Fetching 4 instructions (20 bytes) at a time
- Dependent instructions cannot exist in the same packet
- Dependent instructions should wait until previous instruction is complete
- Need register/FU usage checking (scoreboarding)
- Only one load store unit (only one ld/st instruction at a time)
- Pack instructions as tight as possible (greedy)



Exploration of Packetization in IR

Benefits:

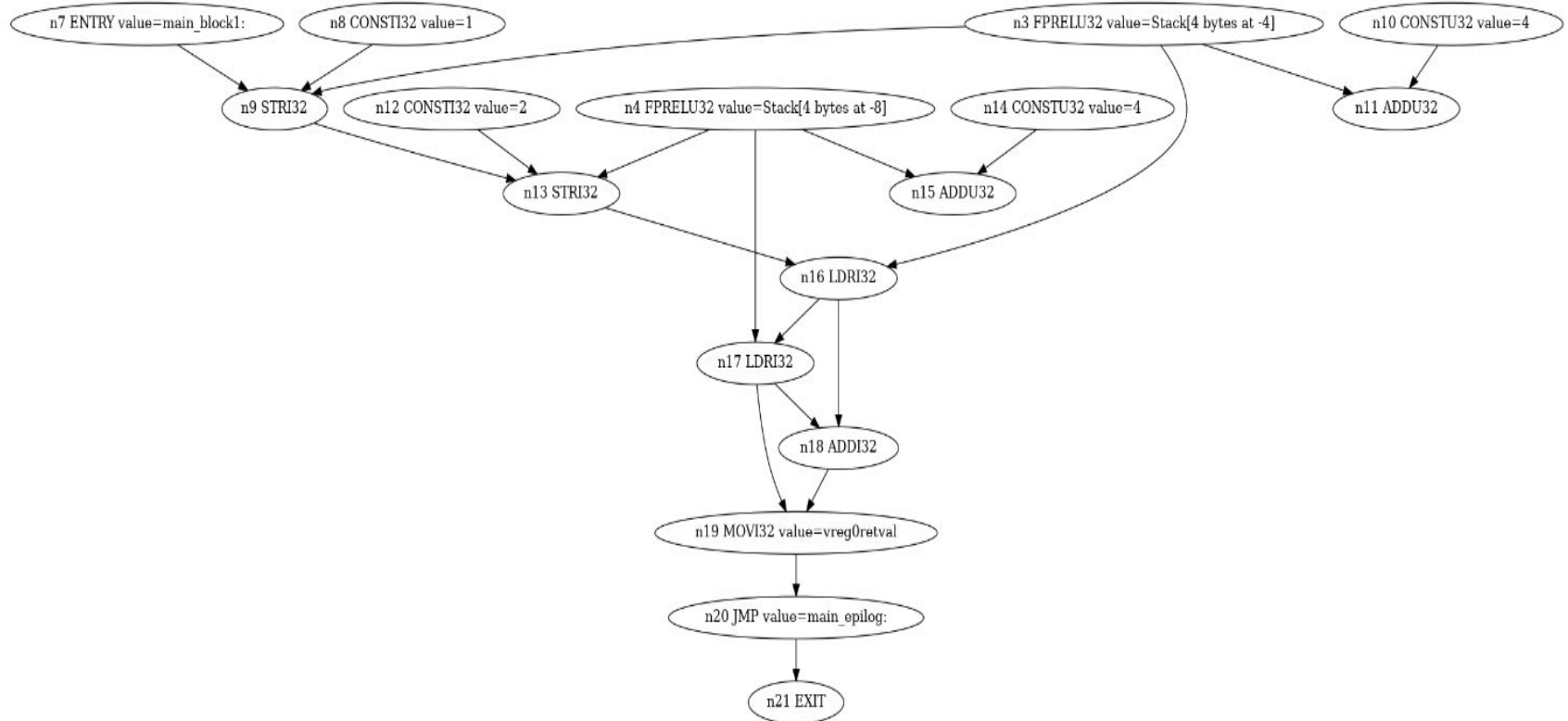
- DAG is automatically generated by PPCI – would simply need to traverse it to packetize
- Higher level code = easier to understand and track instruction dependencies

Drawbacks:

- DAG constructed before register allocation, so unable to track register dependencies (i.e. NOT hardware-accurate)
- Unable to track instruction latency

Conclusion: Packetize after generating assembly

Exploration of Packetization in IR



Packetization Wrapper Pseudocode

```
packetize_basic_block
```

```
Parse assembly → (line, op info) list
```

```
Compute ready_time = build_dependency_graph(...)
```

```
Compute packet indices = greedy_pack(...)
```

```
Convert indices → text packets
```

```
Pad each packet to 4 with "nop"
```

```
Return packets
```

Dependency Tracking Pseudocode

```
build_dependency_graph
```

```
For each instruction i:
    start = 0
    For each source reg s:
        start = max(start, last_write[s])
    If memory op:
        If single LSU: start = max(start, last_mem_cycle + 1)
        If load/store same addr: start = max(start,
last_store_at[mem_key])
    ready_time[i] = start
    last_write[dst] = start + latency(op)
    If store: last_store_at[mem_key] = start + latency(op)
    If mem op: last_mem_cycle = start
Return ready_time
```

Greedy Packing Pseudocode

greedy_pack

```
While there are unscheduled instructions:
    Create empty packet for this cycle
    For each instruction in order:
        Skip if already scheduled or not ready yet
        Skip if control mixes with others
        Skip if memory op and one already in packet
        Skip if RAW/WAR/WAW hazard inside packet
        Add instruction to packet
        Mark as scheduled
        If packet has 4, stop
    If packet empty:
        Record empty packet (bubble → 4 NOPs later)
        Advance cycle
    Else:
        Record filled packet
        Advance cycle
Return list of packets
```

Packetization at Assembly Example Output

```
addi x1, x0, 5
addi x2, x1, 3
mul  x3, x1, x2
sw   x1, 0(x2)
lw   x4, 0(x2)
sub  x2, x4, x5
sub  x4, x3, x2
add  x5, x4, x1
or   x6, x1, x2
L1:
add  x11, x12, x13
add  x12, x13, x14
add  x14, x15, x16
```

Reordering
assembly
instructions



```
Packet 0:
    addi x1, x0, 5
    add  x11, x12, x13
    add  x14, x15, x16
    nop
Packet 1:
    addi x2, x1, 3
    add  x12, x13, x14
    nop
    nop
Packet 2:
    mul  x3, x1, x2
    sw   x1, 0(x2)
    nop
    nop
Packet 3:
    lw   x4, 0(x2)
    nop
    nop
    nop
```

```
Packet 4:
    sub  x2, x4, x5
    nop
    nop
    nop
Packet 5:
    sub  x4, x3, x2
    or   x6, x1, x2
    nop
    nop
Packet 6:
    add  x5, x4, x1
    nop
    nop
    nop
```

```
latency = {"addi": 1, "add": 1,
           "sub": 1, "mul": 3,
           "or": 1, "lw": 3,
           "sw": 1}
```

Assembler/Linker

- PPCI's Base Assembler class can handle any ISA instruction class; outputting instruction streams that can be turned into object files (from assembly code)
- Linker needs to correctly handle relocations (address fixing), map memory to device-accurate address space (in progress)
- Linker outputs binaries (.bin) files that can be fed to device

```
class AtallaAssembler(BaseAssembler):
```

```
ENTRY(_start)
```

```
MEMORY coderam LOCATION=0x80000000 SIZE=0x10000 {  
| SECTION(code)  
}
```

```
MEMORY dataram LOCATION=0x80010000 SIZE=0x10000 {  
| SECTION(data)  
}
```

Example of a memory map

Goals & Next steps

By the end of the semester:

- Fully implement linker in compiler
- Complete ISA patterns for vector instructions to generate assembly
- Integrate packetization scripts into compiler
- Test assembly using simulator
- Final product: working & accurate compiler for all scalar & vector operations.

Next semester:

- Compiler applications (kernels, etc.)

Thank you! Questions?

